

Experiment No. 2

Aim :

To implement Non-Preemptive First Come First Serve (FCFS) CPU Scheduling Algorithm and to evaluate the average turnaround time and average waiting time for a given problem.

Theory :

CPU Scheduling

CPU Scheduling is the mechanism by which the operating system decides which process in the ready queue is allocated the CPU next. Since multiple processes compete for a single CPU, an efficient scheduling policy is required to maximise CPU utilisation, throughput, and fairness, while minimising waiting time, turnaround time, and response time of processes.

First Come First Serve (FCFS)

First Come First Serve is the simplest CPU scheduling algorithm. Processes are allocated the CPU strictly in the order in which they arrive in the ready queue, that is, on a first-come, first-served basis. It is implemented using a simple FIFO (First In First Out) queue: when a process enters the ready queue, its Process Control Block (PCB) is linked onto the tail of the queue, and when the CPU becomes free, it is allocated to the process at the head of the queue.

Non-Preemptive Nature

FCFS is inherently a non-preemptive scheduling algorithm. Once the CPU has been allocated to a process, that process holds the CPU until it voluntarily releases it, either by completing its execution or by requesting an I/O operation. The CPU is never forcibly taken away from a running process, regardless of the arrival of new, possibly shorter, processes.

Turnaround Time and Waiting Time

Turnaround Time of a process is the total time taken from the moment the process arrives in the ready queue to the moment it finishes execution. It is calculated as: $\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$.

Waiting Time of a process is the total time the process spends waiting in the ready queue for the CPU. It is calculated as: $\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$.

The average turnaround time and average waiting time are obtained by summing the respective values for all processes and dividing by the total number of processes. These two metrics are commonly used to judge the efficiency and fairness of a scheduling algorithm.

Convoy Effect

A major drawback of FCFS is the convoy effect. If a process with a very long burst time arrives first (or gets the CPU first), all the shorter processes that arrive after it must wait for the long process to finish, even though they could have completed much faster if scheduled earlier. This significantly increases the average waiting time of the system, especially when short and long processes are mixed.

Advantages and Disadvantages

Advantages: FCFS is simple to understand and implement, requires minimal scheduling overhead, and is inherently fair in the sense that processes are served strictly in the order of their arrival.

Disadvantages: It suffers from the convoy effect, generally results in a higher average waiting time compared to algorithms such as Shortest Job First, and is not suitable for time-sharing or interactive systems since it does not support preemption.

Code

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;

int main(){
    vector<pair<int,int>> process;

    int n;
    cout<<"Enter How many process you have to execute: ";
    cin>>n;

    for(int i = 0; i<n; i++){
        int arrival_time, burst_time;
        cout<<"Enter Arrival Time and Burst Time for Process "<<i+1<<": ";
        cin>>arrival_time>>burst_time;
        process.push_back(make_pair(arrival_time, burst_time));
    }

    sort(process.begin(), process.end());

    for(int i = 0; i<n; i++){
        cout<<"Process "<<i+1<<": Arrival Time = "<<process[i].first<<", Burst Time = 
"<<process[i].second<<endl;
    }
    vector<int> completion_time(n,0);
    vector<int> tat(n,0);
    vector<int> waiting_time(n,0);
    for(int i = 0; i<n; i++){
        if(i==0){
            completion_time[i] = process[i].first + process[i].second;
        }
        else{
            completion_time[i] = max(completion_time[i-1], process[i].first) + process[i].second;
        }
        tat[i] = completion_time[i] - process[i].first;
        waiting_time[i] = tat[i] - process[i].second;
    }

    //Printing the table:
```

```

    cout<<"\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting
Time"<<endl;
    for(int i = 0; i<n; i++){
        cout<<i+1<<"\t"<<process[i].first<<"\t"<<process[i].second<<"\t"<<completion_tim
e[i]<<"\t"<<tat[i]<<"\t"<<waiting_time[i]<<endl;
    }

    //Gantt Chart
    cout<<"\nGantt Chart: "<<endl;
    for(int i = 0; i<n; i++){
        cout<<"| P"<<i+1<<" ";
    }
    cout<<"|"<<endl;
    cout<<"0  ";
    for(int i = 0; i<n; i++){
        cout<<completion_time[i]<<" ";
    }

    //Average Turn Around Time and Waiting Time
    int total_tat = 0;
    int total_waiting_time = 0;
    for(int i = 0; i<n; i++){
        total_tat += tat[i];
        total_waiting_time += waiting_time[i];
    }
    double avg_tat = (double)total_tat / n;
    double avg_waiting_time = (double)total_waiting_time / n;

    cout<<"\nAverage Turnaround Time: "<<avg_tat<<endl;
    cout<<"Average Waiting Time: "<<avg_waiting_time<<endl;

    return 0;
}

```

Output

```
PS C:\Users\lenovo\Desktop\Development\OS Programs> cd "c:\Users\lenovo\Desktop\Development\OS Programs\" ; if ($?) { g++ fcfs.cpp -o fcfs } ; if ($?) { .\fcfs }
Enter How many process you have to execute: 4
Enter Arrival Time and Burst Time for Process 1: 0 2
Enter Arrival Time and Burst Time for Process 2: 3 8
Enter Arrival Time and Burst Time for Process 3: 4 7
Enter Arrival Time and Burst Time for Process 4: 5 6
Process 1: Arrival Time = 0, Burst Time = 2
Process 2: Arrival Time = 3, Burst Time = 8
Process 3: Arrival Time = 4, Burst Time = 7
Process 4: Arrival Time = 5, Burst Time = 6

Process Arrival Time    Burst Time    Completion Time Turnaround Time Waiting Time
1      0              2              2              2              0
2      3              8              11             8              0
3      4              7              18             14              7
4      5              6              24             19             13

Gantt Chart:
| P1 | P2 | P3 | P4 |
0   2  11  18  24
Average Turnaround Time: 10.75
Average Waiting Time: 5
```

Viva Questions

Q1. What is CPU scheduling and why is it needed?

Ans. CPU scheduling is the process by which the operating system selects which process, among those in the ready queue, is given control of the CPU next. It is needed because a single CPU can execute only one process at a time, so a scheduling policy is required to decide the order of execution and make efficient use of the CPU.

Q2. Why is FCFS called a non-preemptive algorithm?

Ans. In FCFS, once a process is allocated the CPU, it is allowed to run to completion (or until it requests I/O) without being interrupted, even if a process with a shorter burst time arrives afterwards. Since the CPU is never forcibly taken away from a running process, FCFS is classified as non-preemptive.

Q3. What is the convoy effect, and how does it affect FCFS?

Ans. The convoy effect occurs when a process with a long burst time occupies the CPU first, forcing all shorter processes behind it in the queue to wait much longer than necessary. This increases the average waiting time significantly and is one of the main drawbacks of the FCFS algorithm.

Q4. How are turnaround time and waiting time calculated in FCFS?

Ans. Turnaround Time is calculated as Completion Time minus Arrival Time, representing the total time a process spends in the system. Waiting Time is calculated as Turnaround Time minus Burst Time, representing the time a process spends waiting in the ready queue before it starts executing.

Q5. What are the main advantages and disadvantages of FCFS scheduling?

Ans. The main advantages of FCFS are its simplicity, ease of implementation, and fairness in serving processes strictly in order of arrival. Its main disadvantages are the convoy effect, a generally higher average waiting time compared to algorithms like Shortest Job First, and its unsuitability for interactive or time-sharing systems due to the lack of preemption.